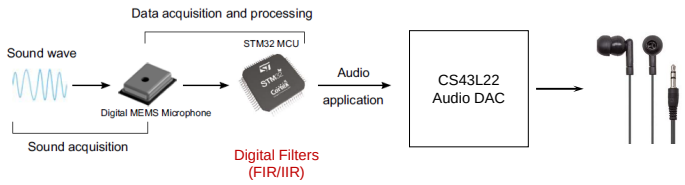
Lab 5

Digital Filters

Ira A. Fulton Schools of Engineering
Arizona State University

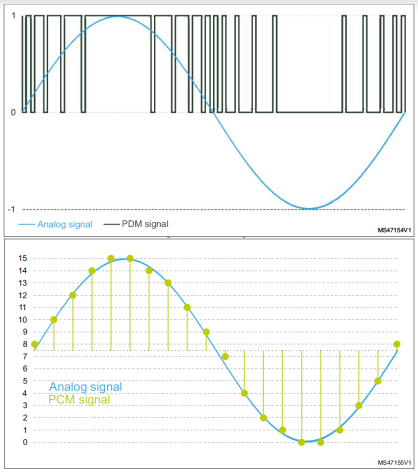
Audio Signal Processing



AN5027 Application note: Interfacing PDM digital microphones using STM32 MCUs and MPUs

PDM and PCM Signals

- Need to convert Pulse Density Modulation (PDM) signals acquired from microphone to Pulse Code Modulation (PCM) signals.



I²S

- I²S is a synchronous serial communication interface.
- It support four different audio standards including the I²S Philips standard, the MSB- and LSB-justified standards, and the PCM standard.
- It can operate as a slave or a master device in full-duplex mode or in half-duplex mode (as a transmitter or a receiver). Master clock can be provided by the interface to an external slave component when the I²S is configured as the communication master.
- Two instances of I²S are used: 1) communicate with microphone (as a receiver) and 2) communicate with audio DAC (as a transmitter).
- It offers Direct Memory Access (DMA) capability for data transmission and reception.
- Details are in “STM32F407 Reference Manual: Serial Peripheral Interface (SPI)”.

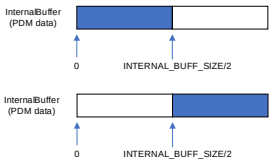
Direct Memory Access (DMA)

- **Direct memory access (DMA) is used to provide high-speed data transfer between peripherals and memory, and between memory and memory.**
- **Data can be quickly moved by DMA without any CPU action. This keeps CPU resources free for other operations.**

How to use DMA?

- **Initialize DMA.**
- **Start DMA.**
- **Implement interrupt callback functions to retrieve/reload data.**
  - DMA interrupts can be produced when
    - Half-transfer is reached.
    - Transfer is complete.
- **Details are in "STM32F407 Reference Manual: DMA Controller".**

Microphone to Memory DMA



- **Initialize: BSP_AUDIO_IN_Init()**
- **Start: BSP_AUDIO_IN_Record()**
- **Callbacks:**
  - When the first half of buffer is filled with microphone PDM data,
    - BSP_AUDIO_IN_HalfTransfer_CallBack() is called.
  - When the second half of buffer is filled with microphone PDM data,
    - BSP_AUDIO_IN_TransferComplete_CallBack() is called.
  - In the callback functions, process the data.

Memory to Audio DAC DMA

- **Initialize: BSP_AUDIO_OUT_Init()**
- **Start: BSP_AUDIO_OUT_Play()**
- **Callbacks:**
  - No need to implement, let audio DAC continuously play the circular buffer.
- **Need to put processed data in the circular buffer to be played.**

Timing Considerations

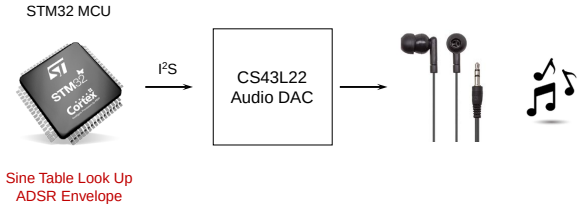
- Microphone and I²S are set up such that, every 1ms, one callback function (half or complete transfer) is called, PDM data is converted to PCM data, 16 samples of 16-bit of microphone audio data becomes available for processing. Therefore, the input sampling rate is 16K samples per second.
- Within 1ms, all signal processing needs to done to meet the real time constraint, including converting PDM to PCM and filter computations.
- The processed audio is saved into AudioOutBuffer to be played by audio DAC at 16K samples per second.

Lab 6

Music Note Synthesis

Ira A. Fulton Schools of Engineering
Arizona State University

Music Note Synthesis



Recall: How to use DMA?

- Initialize DMA.
- Start DMA.
- Implement interrupt callback functions to retrieve/reload data.
  - DMA interrupts can be produced when
    - Half-transfer is reached.
    - Transfer is complete.
- Details are in “STM32F407 Reference Manual: DMA Controller”.

Memory to Audio DAC DMA

- **Initialize: BSP_AUDIO_OUT_Init()**
- **Start: BSP_AUDIO_OUT_Play()**
- **Callbacks:**
  - When the first half of buffer has been played,
    - BSP_AUDIO_OUT_HalfTransfer_CallBack() is called.
    - Fill the first half of buffer with sine table lookup values.
  - When the second half of buffer has been played,
    - BSP_AUDIO_OUT_TransferComplete_CallBack() is called.
    - Fill the second half of buffer with sine table lookup values.
- **Circular AudioOutBuffer is played continuously by audio DAC at 16K samples per second.**

Timing Considerations

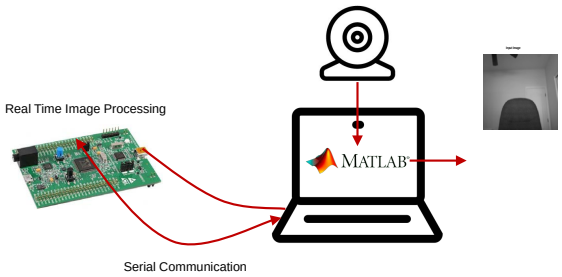
- **Given a 16K sampling rate, after 512 samples stored in half of AudioOutBuffer are played, one callback function (half or complete transfer) is called every 512/16000 = 32ms.**
- **During this time, 512 music audio samples need to be generated and put into AudioOutBuffer to be played next.**

Project 1

Real Time Image Processing

Ira A. Fulton Schools of Engineering
Arizona State University

Real Time Image Processing



Project 2

Applications of Fast Fourier Transform

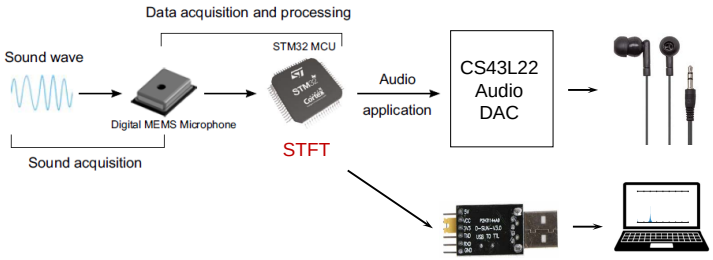
Ira A. Fulton Schools of Engineering
Arizona State University

CMSIS-DSP

- **A suite of common signal processing functions for use on Cortex-M and Cortex-A processor-based devices**
- **The library is divided into different categories such as**
  - Basic math functions
  - Complex math functions
  - Filtering functions
  - Transform functions
  - Etc.
- **A precompiled library file is used in the project:**
  - ARM CMSIS Version 4.5.0
  - CMSIS-DSP V1.4.5 b

https://arm-software.github.io/CMSIS-DSP/latest/

Real Time Spectrum Analyzer

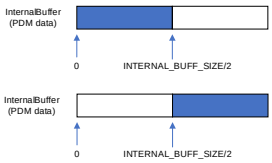


AN5027 Application note: Interfacing PDM digital microphones using STM32 MCUs and MPUs

Recall: How to use DMA?

- **Initialize DMA.**
- **Start DMA.**
- **Implement interrupt callback functions to retrieve/reload data.**
  - DMA interrupts can be produced when
    - Half-transfer is reached.
    - Transfer is complete.
- **Details are in "STM32F407 Reference Manual: DMA Controller".**

Microphone to Memory DMA



Initialize: BSP_AUDIO_IN_Init()

Start: BSP_AUDIO_IN_Record()

Callbacks:

- When the first half of buffer is filled with microphone PDM data,
  - BSP_AUDIO_IN_HalfTransfer_CallBack() is called.
- When the second half of buffer is filled with microphone PDM data,
  - BSP_AUDIO_IN_TransferComplete_CallBack() is called.
- In the callback functions, process the data.

Memory to Audio DAC DMA

Initialize: BSP_AUDIO_OUT_Init()

Start: BSP_AUDIO_OUT_Play()

Callbacks:

- No need to implement, let audio DAC continuously play the circular buffer.

Need to put audio data in the circular buffer to be played.

Memory to UART DMA

Initialize: HAL_UART_Init()

Start: HAL_UART_Transmit_DMA()

Callbacks:

- No need to implement, transmit data when it is ready.

Need to put FFT data in the buffer to be transmitted.

Timing Considerations

Microphone and I^2S are set up such that, every 1ms, one callback function (half or complete transfer) is called, PDM data is converted to PCM data, 16 samples of 16-bit of microphone audio data becomes available.

The samples are accumulated into a block of 1024 samples every (1024 samples) / (16 samples / ms) = 64ms.

A FFT of size 1024 is then computed.

Serial transmission is initiated to transmit

- 512 spectrum data + 1 start flag

Serial transmission time:

- (513*8 bits) / (115200 bits/second) = 36ms

EEE404/591 Real Time DSP
Project 2: Applications of Fast Fourier Transform

I. Objectives

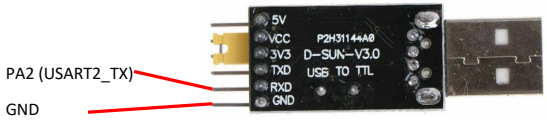
The objective of this lab is to apply FFT algorithm to signal processing applications. Two applications will be implemented:

- Vowel analysis: identify pitch, i.e., fundamental frequency and estimate formant frequencies
- Real time spectrum analyzer: display in real-time the spectrum of the audio captured by the microphone MP45DT02 and investigate the effect of windowing

II. Hardware Setup

Set up the following hardware:

- STM32F407G-DISC1 kit: When you are ready to upload code to your board, connect USB Type-A end of the cable to your computer's USB port, connect the Mini-B end of the cable to the board. The PWR red LED will turn on.
- Use a jumper wire to connect PE14 and PB10 to supply a clock to the microphone.
- For real time spectrum analyzer, serial communication between your computer and microcontroller board needs to be established. Make the following connections between the STM32F407 board header pins PA2, GND, and the serial/USB converter:



III. Software Setup

- MATLAB: prototype algorithms, establish serial communication between computer and board, receive FFT computed by the board and plot for viewing
- Audacity (or any other audio recording software): https://www.audacityteam.org/download/ record vowel sounds
- Online tone generator: https://www.szynalski.com/tone-generator/ verify results of the real time spectrum analyzer

IV. Code

Download the code zip file from Canvas.

V. References

The following documents are posted on Canvas.

- STM32CubeIDE User Guide
- STM32F407 Discovery Board User Manual

VI. Tasks

1. Vowel analysis

1) Record vowel sounds

Audacity will be used to record you speaking two vowels: "a" and "i". An example of each is posted on Canvas. You need to hold the sound for about 5 seconds, which will be used for both MATLAB analysis and with the microcontroller program. Follow instructions in the video and save them as ".wav" files.

2) Analyze vowel sounds using MATLAB

The algorithm you will implement on the microcontroller is prototyped in MATLAB, and the script is provided to you. Run the script and fill out the following table. The first three items in the table can be obtained from the FFT MATLAB plot. Move your cursor close to the peak to display the data point value. The last item in the table is from the MATLAB command window.

Vowel	1st Formant Frequency (Hz)	2nd Formant Frequency (Hz)	Pitch/Fundamental Frequency (Hz)	Calculated Pitch/Fundamental Frequency (Hz)	MATLAB Plot
a					
i					

An example of vowel sounds and formant frequencies is given at https://linguistics.ucla.edu/people/hayes/103/Charts/VChart/. Note the format frequencies vary from person to person due to gender, accent, etc. Don't be surprised to get different numbers from the website.

3) Analyze vowel sounds using the microcontroller

For the vowel analysis algorithm, you need to perform the following task in "main.c":

- Write C function "max_index"

Compare your result from the microcontroller with that from MATLAB. Make sure the FFT forward transform and inverse transform results, pitch index and pitch frequency are the same.

2. Real time spectrum analyzer

Audio samples will be acquired by the MP45DT02 microphone. The microcontroller will compute Short Time Fourier Transform (STFT) of the audio samples using the FFT algorithm. The computed FFT result will be sent to the computer to be displayed by MATLAB via serial communication. At the same time, the acquired audio samples are sent to CS43L22 audio DAC on both left and right channels. You can hear the microphone samples on your headphones or speaker by connecting to the 3.5mm audio jack. Make sure to use a jumper wire to connect PE14 and PB10 to supply a clock to the microphone, and to connect the serial/USB converter properly by following instructions in the "Hardware Setup".

For the real time spectrum analyzer project, you need to perform the following task:

- Fill in the parts labelled as “FILL_IN_BLANK” to use CMSIS FFT functions: one line in “main.c” and three lines in “waverecorder.c”.
- Write C function “apply_window” in “waverecorder.c”.
- Run the algorithm on the microcontroller board with MATLAB script running on the computer. Go to online tone generator: https://www.szynalski.com/tone-generator/. Try different tone frequencies and observe MATLAB plot. Does the FFT result reflect the generated tone frequency?
- Choose one frequency from the online tone generator. Apply different window functions and fill in the table below to document your observation of the effect of the window function. Rectangular window is equivalent to not applying any window. Take screenshots of the online tone generator browser window on one side showing the tone frequency, and the MATLAB plot window on the other side showing the corresponding FFT. Insert screenshots into the table below.

Window Function	Observation	Tone Frequency and MATLAB Plot
Rectangular		
Hamming		
Hanning		

VII. Deliverables

Start your lab report by including the lab title and your name at the top. Then include the following items in order. Submit your lab report as a single PDF file to Canvas.

1. Tables

Include the filled-out tables from Task 1 2) and Task 2.

2. Screenshots

Include the following screenshots. You need to make sure the screenshots are clearly readable. Highlight the requested parts or values.

For each of the “a” and “i” vowels in Task 1, IDE window screenshots showing “pitch_index” and “pitch_freq” values on the right (one for each vowel), and MATLAB command line output screenshots displaying the pitch index and frequency (one for each vowel).

3. Code

Copy and paste your code using the following template. Indicate the file the code belongs to.

Include code for the following functions and lines:

In main.c:

```
uint32_t max_index(float32_t* x, uint32_t length);
```

In waverecorder.c:

```
void apply_window(float* signal_segment, float* window_function, uint16_t length)
```


In main.c and waverecorder.c:
FILL_IN_BLANK parts (4 lines of code)

Exam 1 Review

Exam 1 is closed book, closed notes except for one 8.5x11 sheet. You can use your calculator. Show all your work including derivations, explanations and final answers on the pages provided. The exam is worth 150 points in total.

Topics

Exam 1 tests materials from Module 1, Module 2 and Module 4. You should review all lecture slides for these modules, along with HW1, HW2, Quiz 1 and this practice exam.

Materials covered include

- ADC/DAC, sampling and aliasing, discrete-time representation
- Analog/digital frequencies and conversion
- LTI system, properties, impulse response, FIR/IIR, system output given input
- Linear convolution, region of support
- Modulo operator, circular flip and circular shift, circular convolution (relation to linear convolution)
- STM32F4 architecture, ARM Cortex-M4 instruction set, addressing mode, subroutine call standard
- Qm.n, Qn 2's complement fixed point number representations, conversion between binary and decimal, data range, precision, quantization error
- Fixed-point digital filtering, quantization noise
- Floating-point number representation: convert a decimal number to single precision floating point representation and vice versa

Practice Exam Problems

1. Consider the following continuous-time signal x(t):

$$x(t) = 1.6\cos(10\pi t)$$

(a) Determine the minimum sampling rate in Hertz needed for the A/D conversion so that the analog signal can later be reconstructed from its samples without loss.
(b) Let the sampling frequency (sampling rate) of x(t) be 20 Hertz. Determine the resulting sampled signal x[n]. What is the digital frequency of x[n]?
(c) Determine the highest precision 2's complement 4-bit Qm.n format that can be used to represent the amplitude of the sinusoid x[n].
(d) What is the value of x[n] for n = 0, 1, 2, 3?
(e) Given a digital filter with impulse response h[0] = 1, h[1] = 0.5, compute the linear convolution and 4-point circular convolution of x[n] and h[n]. Is the result of the circular convolution the same as the linear convolution?
(f) Represent x[n] and h[n] in Q1.2 format. What is the quantization error?
(g) Compute output y[n] using input x[n] linearly convolutes with impulse response h[n] in Q1.2 format. Assume intermediate results can be stored in double precision using 8 bits. Represent y[n] in Q1.2 format. Is there overflow? What is the quantization error?

2. Floating point representation:
   a) Represent the number 0.3125 in the single precision floating point format.
   b) Represent the number 14 in the single precision floating point format.

3. Consider a STM32407VG with the following hexadecimal values stored in registers and memory.

Register	Value	Memory Address	Memory Data
R0	0x01D7 B36E	0x2000 8007	0x12
R1	0x2000 8000	0x2000 8006	0x34
R2	0x2000 8004	0x2000 8005	0x56
R3	0xF012 5634	0x2000 8004	0x78
PC	0x0800 8000	0x2000 8003	0x9A
LR	0x0800 4000	0x2000 8002	0xBC
Label	**Value**	0x2000 8001	0xDE
sum	0x0800 2000	0x2000 8000	0xF0

For each of the following instructions, determine which register(s) and/or memory location(s) have been modified and their updated values using the following format. Assume little endian format. Note: for each instruction, start with the initial values given above.

Modified Locations/Registers	Current Updated Value

a) ORR R0, R0, R3, LSL #4
b) LDRH R0, [R1, #4]!
c) LDRSB R0, [R1], #4
d) STRB R0, [R1, #4]

(Refer to lecture slides and Quiz 1 for more ARM Cortex-M4 instruction set examples and problems.)

Answers:

1

(a)
f_{max} = 10π/2π = 5Hz, Nyquist rate = 2*f_{max} = 10Hz (minimum sampling rate to avoid aliasing)

(b)
Sampling period Ts = 1/fs = 1/20 = 0.05s
$x[n] = x(t)|_{t=0.05n} = 1.6\cos(10\pi 0.05n) = 1.6\cos(0.5\pi n)$
Digital frequency is w = 0.5π radians or F = 0.5π/2π = 1/4 cycles

(c)
Amplitude of x[n] is 1.6.
The range of Qm.n fixed point representation is -2^m ~ $2^m - 2^{-n}$.
Q3.0 range is -2^3 = -8 to 2^3-1 = 7
Q2.1 range is -2^2 = -4 to (2^3-1)/2 = 3.5
Q1.2 range is -2^1 = -2 to (2^3-1)/4 = 1.75 (highest precision format)
So Q1.2 is the 4-bit highest precision Q format that can represent 1.6.

(d)
x[0] = 1.6
x[1] = 1.6cos(0.5π) = 0
x[2] = 1.6cos(π) = -1.6
x[3] = 1.6cos(1.5π) = 0

(e)
Linear convolution:
x[n]: 1.6 0 -1.6 0 h[n]: 1 0.5
h[-n]: 0.5 1 (flip h[n], then shift right)
y[0] = x[0]h[0] = 1.6*1 =1.6
y[1] = x[0]h[1]+x[1]h[0] = 1.6*0.5 = 0.8
y[2] = x[1]h[1]+x[2]h[0] = -1.6*1 =-1.6
y[3] = x[2]h[1] = -1.6*0.5 = -0.8

Circular convolution (4-point):
x[n]: 1.6 0 -1.6 0 h[n]: 1 0.5 0 0
Circular flip then circular shift right:
$h[(-n)_4]$ = 1 0 0 0.5 $h[(1-n)_4]$ = 0.5 1 0 0 $h[(2-n)_4]$ = 0 0.5 1 0 $h[(3-n)_4]$ = 0 0 0.5 1
Multiply and sum the x and h buffers:
y[0] = 1.6*1 =1.6 y[1] = 1.6*0.5 = 0.8 y[2] = -1.6*1 =-1.6 y[3] = -1.6*0.5 = -0.8

Yes. x[n] has 3 non-zero number, h[n] has 2, then for linear convolution y[n] has length of 3+2-1=4, linear convolution is equal to circular convolution.

(f)
Convert x[n] and h[n] to Q1.2 format:

x[0] = 1.6 *4 = 6.4, round => 6 = 0110, Q1.2 = 01.10 = 6/4 = 1.5
Quantization error = |1.6-1.5| = 0.1
x[1] = 0, Q1.2 = 00.00
Quantization error = 0
x[2] = -1.6 *4 = -6.4, round => -6 = 1010 (2's complement), Q1.2 = 10.10 = -2+0.5 = -1.5
Quantization error = |-1.6-(-1.5)| = 0.1
x[3] = 0, Q1.2 = 00.00
Quantization error = 0

h[0] = 1*4 = 4 = 0100, Q1.2 = 01.00 = 1
Quantization error = 0
h[1] = 0.5*4 = 2 = 0010, Q1.2 = 00.10 = 0.5
Quantization error = 0

(g)
y[0] = x[0]h[0] = 01.10 (Q1.2) * 01.00 (Q1.2) = $6*4/2^4$ = 0001.1000 (Q2.4) = 01.10 (Q1.2) = 1.5
No overflow.
Quantization error = |1.6-1.5| = 0.1

(For illustration purpose, this is how it is implemented: programmer converts x[0] = 1.6 and h[0] = 1 to Q1.2 format, i.e., x[0] = 6 and h[0] = 4. Then all computations are done in integer (fixed point) format, i.e., 4-bit binary integer multiplication is performed, and result is stored in 8 bits, i.e., 0110 * 0100 = 0001 1000. Programmer knows the product is in Q2.4 and needs to be converted back to Q1.2 format, so the binary integer product is arithmetically shifted right by 2, and the least significant 4 bits are saved, i.e., 0001 1000 >> 2 = 0000 0110 = 0110 = 6. Programmer now has the product in Q1.2 format, to get the actual fractional product, needs to convert from binary fixed point back to decimal fractional representation, i.e., 6/4 = 1.5.

Note if double precision is not available for intermediate results, i.e., the product is stored in 4 bits instead of 8 bits, the least significant 4 bits are saved as the product, i.e., 0110 * 0100 = 0001 1000 = 1000, i.e., 6*4 = -8, we have overflow and subsequently wrong result.)

y[1] = x[0]h[1]+x[1]h[0] = 01.10 (Q1.2) * 00.10 (Q1.2) = $6*2/2^4$ = 0000.1100 (Q2.4) = 00.11 (Q1.2) = 0.75
No overflow.
Quantization error = |0.8-0.75| = 0.05

y[2] = x[1]h[1]+x[2]h[0] = 10.10 (Q1.2) * 01.00 (Q1.2) = $-6*4/2^4$ = 1110.1000 (Q2.4) = 10.10 (Q1.2) = -1.5
No overflow.
Quantization error = |-1.6-(-1.5)| = 0.1

y[4] = x[2]h[1] = 10.10 (Q1.2) * 00.10 (Q1.2)) = $-6*2/2^4$ = 1111.0100 (Q2.4) = 11.01 (Q1.2) = -0.75

No overflow.
Quantization error = |-0.8-(-0.75)| = 0.05

2
(a)

0.3125 = $(-1)^0 * 1.25 * 2^{-2} = (-1)^S * 1.M * 2^{E-127}$

So S = 0, E = 125 (8-bit), M = 0.25 (scaled to a 23-bit integer, $0.25*2^{23} = 2^{21}$)

0	0111 1101	010 0000 0000 0000 0000 0000

(b)

14 = $(-1)^0 * 1.75 * 2^3 = (-1)^S * 1.M * 2^{E-127}$

So S = 0, E = 130 (8-bit), M = 0.75 (scaled to a 23-bit integer, $0.75*2^{23} = 2^{22} + 2^{21}$))

0	1000 0010	110 0000 0000 0000 0000 0000

3

a) ORR R0, R0, R3, LSL #4

Modified Locations/Registers	Current Updated Value
R0	0x01F7 F36E

R3 = 0xF012 5634 << 4 = 0x0125 6340

R0 = 0x01D7 B36E | 0x0125 6340

= 0000 0001 1101 0111 1011 0011 0110 1110 |

0000 0001 0010 0101 0110 0011 0100 0000

= 0000 0001 1111 0111 1111 0011 0110 1110 = 0x01F7 F36E

b) LDRH R0, [R1, #4]!

Modified Locations/Registers	Current Updated Value
R0	0x0000 5678
R1	0x2000 8004

The addressing mode is pre-index with update.

R1 + 4 = 0x2000 8000 + 4 = 0x2000 8004 is used as the address to access memory, the 16-bit half word at address 0x2000 8004 is loaded into the lower 16-bit of R0, i.e., R0 = 0x0000 5678 (little endian: lowest address stores least significant byte).

R1 is then updated to 0x2000 8004.

c) LDRSB R0, [R1], #4

Modified Locations/Registers	Current Updated Value
R0	0xFFFF FFF0
R1	0x2000 8004

The addressing mode is post-index.

R1 = 0x2000 8000 is used as the address to access memory, the 8-bit byte at address 0x2000 8000 is loaded into the lower 8-bit of R0, then is sign extended to the rest of the register i.e., R0 = 0xFFF FFF0. (The sign bit of 0xF0 is 1, therefore 1 is sign extended to all the upper bits of the register R0.)

R1 is then updated to 0x2000 8004.

d) STRB R0, [R1, #4]

Modified Locations/Registers	Current Updated Value
0x20008004	0x6E

The addressing mode is pre-index.

R1 + 4 = 0x2000 8000 + 4 = 0x2000 8004 is used as the memory address.

The least significant byte of R0, i.e., 0x6E, is stored at memory address 0x2000 8004.

R1 is unchanged.

Review lecture slides and Quiz 1 for more ARM Cortex-M4 instruction set examples and problems.

EEE 404/591 Real-Time DSP Systems, Spring 2026
EXAM 1

Name:_____**KEY**_____ASU ID:_____

Problem 1:
Consider a STM32407VG with the following hexadecimal values stored in registers and memory.

Register	Value	Memory Address	Memory Data
R0	0xD701 B36E	0x2000 80F7	0x0F
R1	0x2000 80F0	0x2000 80F6	0xE1
R2	0x2000 80F4	0x2000 80F5	0x2D
R3	0xF012 5634	0x2000 80F4	0xAB
R4	0x0000 0001	0x2000 80F3	0x3C
LR	0x0800 8010	0x2000 80F2	0x49
PC	0x0800 4000	0x2000 80F1	0x75
		0x2000 80F0	0x66

For each of the following instructions, determine which register(s) and/or memory location(s) have been modified and their updated values using the following format. Assume little endian format. Note: for each instruction, start with the initial values given above.

Modified Locations/Registers	Current Updated Value

(a) (5 pts) LDR R0, [R1], #4

Modified Locations/Registers	Current Updated Value
R0	**0x3C49 7566**
R1	**0x2000 80F4**

Post index, load word at address R1 to R0, then increment R1 by 4.

(b) (5 pts) STRH R3, [R2, #2]

Modified Locations/Registers	Current Updated Value
0x200080F6	**0x34**
0x200080F7	**0x56**

Pre-index, store the lowest half word of R3 to memory address R2 + 2. R2 is unchanged.

(c) (5 pts) AND R0, R0, R3, LSL #8

Modified Locations/Registers	Current Updated Value
R0	0x1200 3000

R3 = 0xF012 5634 << 8 = 0x1256 3400
R0 = 0xD701 B36E & 0x1256 3400
= 1101 0111 0000 0001 1011 0011 0110 1110 &
0001 0010 0101 0110 0011 0100 0000 0000
= 0001 0010 0000 0000 0011 0000 0000 0000 = 0x1200 3000

(d) (5 pts) BX LR

Modified Locations/Registers	Current Updated Value
PC	0x0800 8010

Branch and exchange, return from a subroutine PC = LR.

Problem 2:
Consider a system S with the following input-output relationship:

$$y[n] = 1.5x[n] - x[n-1] + 0.5x[n-2]$$

Consider the following input:

$$x[n] = \begin{cases} 3, n = 0 \\ 1, n = 1 \\ 2, n = 2 \\ 0, otherwise \end{cases}$$

(a) (5 pts) Determine and plot the impulse response h[n] of the system.
(b) (5 pts) Is this system FIR or IIR? Justify your answer.
(c) (5 pts) Compute the output of the system at n = 2, i.e. y[2].

(a)
By definition, h[n] = 1.5*δ[n] - δ[n-1] + 0.5*δ[n-2]
Show plot: impulse at 0 (magnitude 1.5), impulse at 1 (magnitude -1), impulse at 2 (magnitude 0.5)
(b)
FIR, there is no feedback.
(c)
y[2] = 1.5*x[2] – x[1] + 0.5*x[0] = 1.5*2 - 1*1 + 0.5*3 = 3 - 1 + 1.5 = 3.5

Problem 3:
Consider the following continuous-time signal x(t):

$$x(t) = 2\cos\left(200\pi t + \frac{3\pi}{8}\right)$$

(a) (5 pts) State the Nyquist Theorem.
(b) (10 pts) Determine the minimum sampling rate in Hertz needed for the A/D conversion so that the analog signal can later be reconstructed from its samples without loss.

(c) (10 pts) Let the sampling frequency (sampling rate) of x(t) be 400 Hertz. Determine the resulting sampled signal x[n].

(a)

Sampling frequency has to be greater than twice the signal max frequency to avoid aliasing: $f_s > 2f_M$

(b)

$\omega_M = 200\pi$, $f_M = 100$Hz

Nyquist rate = $2\omega_M = 400\pi$

Sampling rate $\omega_s > 400\pi$, i.e. $T_s < 2\pi/(400\pi) = 0.005$sec, i.e., $f_s > 200$Hz

(c)

For $f_s = 400$Hz, Ts = 0.0025s

$$x[n] = x(t)|_{t=0.0025n} = 2\cos\left(200\pi\frac{1}{400}n + \frac{3\pi}{8}\right) = 2\cos\left(\frac{1}{2}\pi n + \frac{3\pi}{8}\right)$$

Problem 4:
(a) (10 pts) Determine the range of a 2's complement Q2.4 representation. What is the precision of Q2.4 format?

Qm.n range is $-2^{(N-1-n)} \sim 2^{(N-1-n)}-1/2^n$
Q6.0 range is -2^6 = -64 to 2^6-1 = 63
Q2.4 range is -2^6/(2^4) = -64/16 = -4 to (2^6-1)/16 = 63/16 = 3.9375
So range for Q2.4 is -4 to 3.9375.

Precision = 2^(-4) = 0.0625

(b) (15 pts) Find the 2's complement Q3.2 representation of the decimal number -4.1, and determine the resulting quantization error.

Range for Q3.2 is -32/4 to 31/4, i.e., -8 to 7.75, so fit in the range.

Integer value: R(-4.1*4) = R(-16.4) = -16 = -(010000) = 110000

Q3.2 = 1100.00= -16/4 = -4

Error = |-4.1-(-4)| = 0.1

(c) (10 pts) Determine the decimal value of the Q3.4 2's complement number 1001 0101.

Integer value = 1001 0101 **= -(0110 1010 + 1) = -(0110 1011) = -107**

Q3.4 = -107/(2^4) = -6.6875

Problem 5:

(a) (10 pts) Represent the number 0.5625 in the single precision floating point format.
(b) (10 pts) Represent the number 5 in the single precision floating point format.

(a)
$0.5625 = (-1)^0 * 1.125 * 2^{-1} = (-1)^S * 1.M * 2^{E-127}$
So S = 0, E = 126 (8-bit), M = 0.125 (scaled to a 23-bit integer, $0.125*2^{23} = 2^{20}$)

0	**0111 1110**	**001 0000 0000 0000 0000 0000**

(b)
$5 = (-1)^0 * 1.25 * 2^2 = (-1)^S * 1.M * 2^{E-127}$
So S = 0, E = 129 (8-bit), M = 0.25 (scaled to a 23-bit integer, $0.25*2^{23} = 2^{21}$)

0	**1000 0001**	**010 0000 0000 0000 0000 0000**

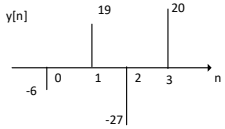
Problem 6:

Consider the finite-length signals x[n] and h[n] given by:

$$x[n] = \begin{cases} 2, n = 0 \\ -3, n = 1 \\ 4, n = 2 \end{cases} \text{ and } h[n] = \begin{cases} -3, n = 0 \\ 5, n = 1 \end{cases}$$

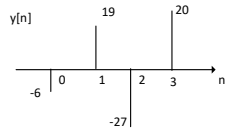
(a) (15 pts) Compute and plot y[n], the 4-point circular convolution of x[n] and h[n].
(b) (15 pts) Compute and plot y'[n], the linear convolution of x[n] and h[n].
(c) (5 pts) Is the result of the circular convolution the same as the linear convolution of x[n] and h[n]? Explain.

(a)
Circular convolution
x[n]: 2 -3 4 0 h[n]: -3 5 0 0
$h[(-n)_4]$ = -3 0 0 5 y[0] = 2*(-3) = -6
$h[(1-n)_4]$ = 5 -3 0 0 y[1] = 2*(5) + (-3)*(-3) = 10+9 = 19
$h[(2-n)_4]$ = 0 5 -3 0 y[2] = (-3)*(5) + 4*(-3) = -15 + -12 = -27
$h[(3-n)_4]$ = 0 0 5 -3 y[3] = 4*5 = 20



(b)
Linear convolution

y[0] = x[0]h[0] = 2*(-3) = -6
y[1] = x[0]h[1]+x[1]h[0] = 2*(5) + (-3)*(-3) = 19
y[2] = x[1]h[1]+x[2]h[0] = (-3)*(5) + (4)*(-3) = -27
y[3] = x[2]h[1] = (4)*(5) = 20



(c)
The circular convolution is the same as the linear convolution, since circular convolution N = 4 ≥ length of linear convolution, i.e., lx+lh-1 = 3+ 2–1 = 4.